

EEG FIELD KIT — CONNECTIVITY TEST PROGRAM

TOOL-EEG-022 Rev A · functional specification, technical description and user manual

Table of Contents

Document: TOOL-EEG-022 **Revision:** A **Date:** 1 September 2026 **Governing documents:** DSN-EEG-003 Rev C, then RFQ-EEG-001 Rev E, then FW-EEG-001 Rev C section 5, which is normative for the wire format. **Licence:** source MIT, this document CC BY-SA 4.0. **Status:** the tool is written and its logic is tested against a simulated device. **It has never been run against real hardware, because no unit has been built.**

Part 0 — What this is, in one paragraph

A single HTML file. You double-click it, it opens in Chrome or Edge, you plug the EEG instrument into a USB port, press one button, and it tells you whether the computer and the instrument can talk to each other. It takes no measurement, touches no electrode, starts no recording and contacts no server. It exists so that when something does not work, you can find out in thirty seconds whether the problem is the cable, the computer, the browser or the firmware — before anyone starts blaming the electronics or the study protocol.

Part 1 — Functional specification

1.1 Why it exists

The end architecture of the study is:

EEG instrument —USB→ participant's computer —WebSocket→ TI
One Voice server
(Chromium browser)

Three links, three ways to fail, and only one of them is the instrument. When a participant says “it does not work”, the programme needs to know which link broke without shipping the kit back. This tool tests the first link on its own, in isolation, on the participant's own computer.

It is also the first thing a manufacturer runs after assembling a unit, before the full production test of TST-EEG-004: if TOOL-EEG-022 fails, nothing in TST-EEG-004 will pass and there is no point starting it.

1.2 What it must do (requirements)

#	Requirement	Level	Verified by
T-01	Run in a Chromium browser on Windows and macOS with no installation, no server, no compiler and no administrator rights	M	Part 3, step 1
T-02	Open the instrument over Web Serial using the same code path the study runner will use	M	shared <code>transport.js</code> , step D1
T-03	Confirm the tool's own protocol implementation before testing anything else	M	steps S1–S3
T-04	Read the device's identity — serial, firmware, board revision, capabilities — without starting a session	M	step D1, <code>CMD_IDENTIFY</code>
T-05	Prove the full frame path in both directions: COBS, CRC-32, both USB endpoints, the ack path	M	steps D4–D6, <code>CMD_LOOPBACK</code>
T-06	Detect a transport that corrupts particular byte values	M	step D6, 256-value sweep
T-07	Measure command round-trip latency	S	step D7
T-08	Confirm the device is silent when no session is running	M	step D8
T-09	Take no measurement, energise no electrode, write nothing to the microSD card	M	Part 2.6
T-10	Contact no network endpoint of any kind	M	<code>build.py</code> refuses to emit a file with an external reference
T-11	Produce a saved report an operator can attach to an email or a device history record	S	Part 3, step 6

#	Requirement	Level	Verified by
T-12	Share its protocol, transport and sink code with the production study runner, with no second implementation of the wire format	M	Part 2.2

1.3 What it deliberately does not do

- **It does not measure anything.** No channel is decoded, no impedance is checked, no electrode is read. A pass says the link works; it says nothing about signal quality, electrode contact, noise, or whether the instrument is safe to put on a person.
 - **It does not connect to the server.** There is no network code in the delivered file, and the build refuses to produce one that has any.
 - **It does not write to the device.** The two commands it sends are read-only by construction: one asks for an identity, one asks for an echo. Nothing is provisioned, configured or stored.
 - **It is not a substitute for TST-EEG-004.** It is one page of the production test's first step, made runnable on its own.
-

Part 2 — Technical description

2.1 Files

webtest/ EEG-Connectivity-Test.html file, ~48 kB. index.html (development) build.py js/ protocol.js framing, sequence tracking transport.js round trips sinks.js production upload sink diagnostics.js ui.js tests/ protocol.test.mjs tests/...) roundtrip.test.mjs device, in JS interop/ firmware run.sh harness.c ESP-IDF stubs drivers_sim.c drivers.h host.mjs	THE DELIVERABLE. One self-contained the same page, loading the modules below inlines the modules into the deliverable COBS, CRC-32, the 10-byte header, Web Serial port, read loop, command/ack the FrameSink interface, and the the test sequence S1-S5 and D1-D8 the screen. Small on purpose COBS, CRC, framing, sequence wrap (node the whole exchange against a simulated the same exchange against the REAL sh webtest/tests/interop/run.sh compiles firmware/main/main.c against simulated peripherals implementing drives it with this tool's own
--	---

protocol.js
stubs/ the minimum FreeRTOS/TinyUSB/driver
headers to compile

Why interop/ exists, when roundtrip.test.mjs already tests the exchange. That test drives the real host code against a device simulated *in JavaScript*, written by hand from the same specification as main.c. Both drifted from that specification, independently and in different directions – and every JS test passed throughout, because the JS host and the JS device shared one misunderstanding. interop/ removes the shared author: it compiles the actual firmware translation unit, wires stdin and stdout to the simulated USB endpoints, and drives it with the actual protocol.js. It needs a C compiler and Node, no ESP-IDF and no hardware, and it runs 32 checks in a couple of seconds. It found the defect that made every command from this tool start a recording session.

2.2 Architecture, and why the tester is production code

The tool is **not** a throwaway page. Everything except ui.js and diagnostics.js is the browser client the study will use, written once:

normative	protocol.js COBS · CRC-32 · frames · seq	shared,
	transport.js Web Serial · command/ack	shared
	sinks.js FrameSink interface └─ InspectorSink ← this tool └─ UploadSink ← the study runner	shared

FrameSink is the seam. A sink obeys three rules, and each comes from a requirement rather than from taste:

1. **accept() must not block the read loop.** At 1000 Hz a frame arrives every 20 ms and USB will not wait. A sink that needs time queues and returns.
2. **A sink must be able to report backpressure.** F-06's ring buffer and CMD_RETRANSMIT exist so that a host which falls behind can catch up rather than lose data. A sink that cannot say "I am behind" makes both useless.
3. **A sink must never silently drop.** F-07: silent loss is not permitted. If it discards, it counts, and the count is visible.

UploadSink — the server path — is written, commented and **shipped unused**. It is not called by this tool and connects to nothing. It is included because writing it is what proves the interface is the right shape, and because the four things it needs from the server are the four things easiest to get wrong when the server is designed: an ordered framed channel rather than a series of POSTs; backpressure the client can observe; an ack carrying the highest contiguous sequence durably stored; and a resume handshake so a dropped socket does not lose a block. **It has never been tested against a server, because no server endpoint exists.**

2.3 The two firmware commands

Added to FW-EEG-001 for this tool. Neither touches the converters, the electrodes, the codec or the card, so both are safe on a bare bench unit.

Opcode	Name	Request	Reply
0x0F	CMD_IDENTIFY	none	14 bytes + NUL-terminated serial: proto version, firmware major/minor, board revision letter, ring bytes (u32 LE), capability flags (u32 LE), current rate code, number of supported rates
0x10	CMD_LOOPBACK	up to 240 bytes	the same bytes, unchanged

Both results arrive in the acknowledgement envelope of FW-EEG-001 section 6.2, not bare. The CMD_ACK payload is [0] the opcode echoed, [1] reserved zero, [2] the status code, [3] the result length, [4..] the result. `parseAck()` in `js/protocol.js` is the only place that knows this; `identify()` and `loopback()` read `ack.result`.

Corrected 2026-09-02. The firmware, this tool and the specification had three different answers to this question. FW-EEG-001 section 6.2 has always specified the envelope above. `main.c` emitted { opcode, status } with the status at offset 1 and no length, and CMD_IDENTIFY, CMD_LOOPBACK and CMD_CLOCK_XCHG bypassed even that and returned their result at offset 0. This tool read the identity struct from offset 0, so it agreed with the firmware by accident on one command and with nothing on the others. All three now agree, and `webtest/tests/interop/` exists to keep them that way – it compiles the real `main.c` and drives it with the real `protocol.js`, which is the only test here that can catch the two sides drifting together.

The echo matters beyond tidiness. `transport.js` matches each reply to the command that asked for it **by opcode**. Before the echo existed on every reply it matched by arrival order, so one command timing out made every later reply answer the wrong promise.

Capability flags: bit 0 CDC, 1 WebUSB, 2 microSD mounted, 3 codec initialised, 4 ATECC present, 5 provisioned (config zone locked).

Corrected 2026-09-02. Bit 3 was documented here and displayed by this tool, and `main.c` never set it under any condition, so a working codec always read as absent. Bit 4 was set only when the ATECC's configuration zone was **locked**, which conflates “the part is fitted” with “the unit has been provisioned” – so a good board coming off the line reported no secure element at all, at exactly the moment a production tester is asking whether the part is there. Bits 3 and 4 now mean what this table says they mean, and bit 5 alone carries provisioning.

CMD_LOOPBACK is the more valuable of the two. CMD_IDENTIFY proves the device answers; the loopback proves the **whole path** — COBS stuffing, the CRC, the sequence counter, both USB endpoints and the ack dispatch — in one round trip, with a payload the host chose.

2.4 The test sequence

Self-checks, no device required. If any of these fails, the tool or the browser is at fault and no conclusion may be drawn about a unit.

#	Check	Fails when
S1	CRC-32 against the standard check value <code>crc32("123456789") = 0xCBF43926</code>	the implementation is wrong
S2	COBS round trip over four payloads, including all-zero and a run longer than 254 bytes	the stuffing is wrong
S3	A frame this tool builds is one this tool can parse	the two halves disagree
S4	<code>navigator.serial</code> exists	Safari, Firefox, or a mobile browser
S5	The page is a secure context	served over plain <code>http://</code> from somewhere other than localhost

Device checks.

#	Check	What a failure means
D1	The device answers <code>CMD_IDENTIFY</code>	no firmware, wrong firmware, or the cable is charge-only
D2	Protocol versions match	one side is older; do not interpret the frames
D3	The unit serial is provisioned	<i>warning only.</i> End-of-line provisioning has not run. The link is fine; the browser cannot bind a persistent authorisation to this unit (F-04)
D4	Loopback, 5 bytes including <code>0x00</code> and <code>0xFF</code>	basic framing
D5	Loopback, a full 240-byte payload	buffer sizing at both ends
D6	Loopback sweep across all 256 byte values	a transport that mangles particular values — a COBS bug, or a driver doing newline translation
D7	Round-trip latency, 20 commands, median and p95	<i>warning above 50 ms.</i> Suspect a hub, a virtual COM driver, or a busy machine. Matters because the stimulus timing of E-13 depends on it
D8	The link is quiet for one second	<i>warning if not.</i> A session is already running, or the firmware streams without being asked

Frame integrity, counted over everything received since the port opened: bytes in, frames decoded, CRC errors, version errors, short frames, resyncs, oversize discards, sequence gaps, frames missing. On a healthy link every error row is zero.

2.5 Why one file, and why `file://` works

Web Serial requires a *secure context*. Chromium treats `file://` as potentially trustworthy, so a double-clicked local file qualifies and no server is needed.

Module **files**, however, are blocked by CORS from a `file://` page — a page that does `<script type="module" src="js/ui.js">` shows a blank screen. An **inline** `<script type="module">` has nothing to fetch and runs normally. So the source stays as modules, and `build.py` inlines them into `EEG-Connectivity-Test.html`.

That is also the honest answer to “a compiled runtime for Windows and macOS”: there is nothing to compile. One file, both operating systems, no installer, nothing to sign, nothing to trust beyond a file you can read.

`build.py` will refuse to emit the deliverable if it still references any local file or any network resource, so T-10 cannot be broken by accident.

2.6 Safety

The tool cannot energise anything. It sends two opcodes, neither of which reaches the ADS1299 converters, the bias drive, the audio codec or the microSD card, and the device is battery-powered and galvanically isolated from the computer by the ADuM4160 module (E-24) whether this tool is running or not.

It is nevertheless **not** a substitute for the electrical safety review of RISK-EEG-011 section 7. A passing connectivity test says nothing about patient safety and must never be quoted as if it did.

2.7 What was found while writing it

Writing a host that parses the wire format exposed a firmware defect that no document had caught, now registered as **FW-D19**:

The frame header was copied onto the wire with `memcpy(txbuf, &h, sizeof h)`, and `sizeof(frame_hdr_t)` is **12, not 10**. The struct is tail-padded to its four-byte alignment, so two padding bytes sat between the header and the payload. Every host parser written to FW-EEG-001 section 5.1 — including this package’s own `firmware/tools/verify_stream.py`, which unpacks “<BBHIBB” — would have misparsed every frame by two bytes.

Fixed by serialising the header field by field, with a `_Static_assert` on the ten-byte length. A wire format is a contract with other people’s software and must never be a struct copy.

Part 3 — User manual

What you need

- The EEG instrument and its USB cable.
- A computer running **Windows 10 or later** or **macOS 11 or later**.

- **Google Chrome** or **Microsoft Edge**. Safari and Firefox will not work — they do not implement the browser feature that talks to USB devices, and no setting changes that.
- The file `EEG-Connectivity-Test.html`.

You do not need to install anything, you do not need administrator rights, and you do not need an internet connection.

Step 1 — Open the tool

Double-click **EEG-Connectivity-Test.html**.

If it opens in the wrong browser, right-click it and choose *Open with* → *Google Chrome* (or *Microsoft Edge*).

The page tells you, near the top, which browser it is running in. If it says Safari or Firefox, close it and open it in Chrome or Edge.

Step 2 — Check the tool checks itself

Section 2, *The tool checks itself*, fills in immediately. All five lines should show a green tick. If any shows a red cross, stop: the problem is the browser or the file, not your instrument, and the line tells you which.

Step 3 — Plug in the instrument

Use the supplied USB cable. Connect it to the socket on the pod, and to a USB port **on the computer itself** — not through a hub, a monitor or a docking station, if you can avoid it.

Step 4 — Choose the device

Press **Choose the device**.

Your browser shows a small window listing the USB devices it can see. Choose the EEG instrument and press *Connect*.

- **If the list is empty**, the computer cannot see the instrument. Try a different cable — many USB cables carry power only and no data — then a different port. If it is still empty, the instrument is not powered or its firmware is not running.
- **If you press Cancel**, nothing happens and you can press the button again.

Step 5 — Run the test

Press **Run the connectivity test**. It takes a few seconds.

The bar under the buttons then reads one of:

	What it means	What to do
PASS	The computer and the instrument talk to each other correctly.	Nothing. Carry on.

	What it means	What to do
PASS WITH WARNINGS	The link works, but something is worth knowing.	Read the amber lines. The common one is <i>Unit serial is provisioned</i> — normal on a new unit that has not been through end-of-line provisioning.
FAIL	The link does not work.	Read the red lines; each says what failed. Try steps 3 and 4 again with a different cable and a different port before reporting a fault.

Step 6 — Save the report

Press **Save the report**. Your browser saves a small .json file to your Downloads folder, named after the instrument's serial number and the time.

Attach that file to your email if you are reporting a problem. It contains the test results, the instrument's identity and your browser version — and nothing else. It contains no measurement, no personal data and nothing about you.

If something goes wrong

What you see	What it usually is
The device list is empty	A charge-only USB cable. This is by far the most common cause. Try another cable first.
<i>Browser supports Web Serial</i> is a red cross	You are in Safari or Firefox. Open the file in Chrome or Edge.
<i>Secure context</i> is a red cross	The file is being served from a web address that is not https. Open the file directly instead.
<i>Device answers CMD_IDENTIFY</i> fails	The instrument is powered but its firmware is not answering. Unplug it, wait five seconds, plug it back in. If it still fails, the unit needs service.
<i>Protocol version matches</i> fails	The instrument's firmware and this tool are different versions. Ask the programme for the matching version of one of them.
<i>Round-trip latency</i> is amber	You are probably connected through a hub or a docking station. Plug directly into the computer.
<i>Link is quiet when idle</i> is amber	A recording session is already running on the instrument. Unplug it and plug it back in.

What this test does not tell you

It tells you the computer and the instrument can talk. It does **not** tell you that the electrodes are making good contact, that the signal is clean, that the instrument is calibrated, or that it is safe to wear. Those are separate checks, and a green PASS here says nothing about any of them.

Revision history

Rev	Date	Change
A	1 September 2026	First issue. Written alongside CMD_IDENTIFY and CMD_LOOPBACK, which were added to FW-EEG-001 for it. Found and fixed FW-D19, the padded frame header.